

AI Quant Research Platform

v2.0 集成方案 · 开发任务书

开源框架集成 · 五层数据源接入 · 完整工程落地方案

基于现有仓库 Metroids048/AI- | Phase 0 → Phase 3 完整路线图

零、现状诊断与集成逻辑

0.1 仓库现状快照

通过完整读取仓库文件，当前状态如下：

模块	现状	问题等级
pyproject.toml	依赖完整（ccxt/freqtrade/langchain/vectorbt等）	✔ 无问题
.env.example	仅有 BINANCE + CLAUDE API Key，缺少 60%数据源密钥	⚠ 待补全
docker-compose.yml	只有 postgres/redis/api 三服务，无 celery_worker	⚠ 缺组件
services/ 目录	全空，无任何实现代码	✖ 最紧迫
apps/api/	空壳，无路由无模型	✖ 待实现
AGENTS.md	六层架构约束完整，18 字段定义完整	✔ 优秀
research_source/worldquant_adapter/	目录存在但内容空	⚠ 核心资产待开发
.github/agent/memory/	三个记忆文件是否有内容未知	⚠ AI 协作命脉
tests/	全空	✖ 无测试保护

0.2 集成核心逻辑：框架是工具，平台是容器

这份方案解决的核心问题是：如何让 **Freqtrade**、**Jesse**、**VectorBT**、**vnpy**、**Lean** 等框架不是各自孤立运行，而是作为可插拔模块嵌入你的平台，通过统一接口对外提供服务。

核心原则： 所有开源框架只暴露给 **services/** 层，绝不直接暴露给 **apps/api/** 或 **frontend/**。框架是实现细节，平台定义接口。

集成模式分三类：

- 直接依赖（作为 Python 库调用）：CCXT、VectorBT、pandas-ta、TA-Lib
- 进程封装（作为子进程或 Docker 服务运行）：Freqtrade、Jesse、vnpy
- 数据适配（转换外部数据格式为内部统一模型）：WorldQuant Alpha、GitHub 策略、Telegram 信号

一、整体架构升级方案

1.1 目录结构 v2.0（完整版）

在现有骨架基础上，完整填充每个目录的职责：

根目录新增

<code>docker-compose.yml</code>	← 补充 <code>celery_worker</code> 、 <code>freqtrade</code> 、 <code>grafana</code> 服务
<code>docker-compose.dev.yml</code>	← 开发环境覆盖（热重载、debug 端口）
<code>Makefile</code>	← 统一命令入口（ <code>make up</code> / <code>make backtest</code> / <code>make test</code> ）
<code>.env.example</code>	← 扩充至全部数据源密钥（见第三章）

services/ 目录（核心实现层，全部新建）

<code>services/</code>	
<code>data/</code>	← 数据层：五级数据源统一管道
<code>collectors/</code>	← 各数据源采集器
<code>market/</code>	← A 级：OHLCV、OI、Funding Rate（CCXT 驱动）
<code>macro/</code>	← B 级：FOMC、CPI（经济日历 API）
<code>news/</code>	← C 级：金十、Reuters、CoinDesk RSS
<code>social/</code>	← D 级：Twitter/X、Telegram
<code>research/</code>	← E 级：GitHub、论文、YouTube 摘要
<code>normalizer/</code>	← 统一数据格式转换器
<code>storage/</code>	← TimescaleDB 写入、Redis 缓存管理
<code>risk_classifier/</code>	← 新闻/社媒→风险等级标签
<code>strategy_library/</code>	← 策略知识库
<code>models.py</code>	← Strategy SQLAlchemy 模型（18 字段）
<code>repository.py</code>	← CRUD 操作
<code>importers/</code>	← 各来源策略导入器
<code>worldquant.py</code>	← WQ Alpha → Strategy 对象
<code>github_crawler.py</code>	← GitHub 策略 → Strategy 对象
<code>telegram_parser.py</code>	← 喊单信号 → 统计记录
<code>agents/</code>	← AI Agent 层
<code>base.py</code>	← Agent 基类（Claude API 调用封装）
<code>strategy_agent.py</code>	← 自然语言→规则化策略
<code>coding_agent.py</code>	← 规则→Freqtrade/Jesse 代码
<code>backtest_agent.py</code>	← 触发回测→解析报告
<code>review_agent.py</code>	← 每日日报生成
<code>news_agent.py</code>	← 新闻分类→风险标签
<code>research_agent.py</code>	← 策略研究数据处理
<code>validation/</code>	← 验证层
<code>engines/</code>	
<code>freqtrade_runner.py</code>	← Freqtrade 回测/模拟盘封装
<code>jesse_runner.py</code>	← Jesse 回测封装（交叉验证用）
<code>vectorbt_scanner.py</code>	← 大规模参数扫描

lean_runner.py	← 美股/期货回测（第三阶段）
report_parser.py	← 回测结果→标准化 Report 对象
gate_keeper.py	← 硬性门槛检查（Sharpe/PF/DD）
execution/	← 执行层
engine.py	← 下单/止损/止盈/日志
risk_engine.py	← 实时风控（仓位/回撤/事件）
freqtrade_bridge.py	← 与 Freqtrade 实盘 API 通信
review/	← 复盘层
daily_reporter.py	← 日报生成
failure_analyzer.py	← 失效模式识别
strategy_updater.py	← 回写 failure_reasons/iteration_history

新增独立服务目录

infra/	
freqtrade/	← Freqtrade 实例配置
config/	← config.json 模板
strategies/	← 所有 Freqtrade 策略文件
user_data/	← 回测结果、模拟盘日志
jesse/	
strategies/	← Jesse 策略文件
timescale/	
init.sql	← TimescaleDB 超表初始化 SQL
grafana/	
dashboards/	← 预置策略监控面板

1.2 docker-compose v2.0（完整版）

在原有 postgres/redis/api 基础上补充所有缺失服务：

服务名	镜像/说明	端口	职责
postgres	TimescaleDB （ postgres 超集）	5432	存储 K 线、策略库、回测结果、交易日志
redis	redis:7	6379	实时行情缓存、Celery 任务队列
api	Python FastAPI（本地构建）	8000	REST API 总入口
celery_worker	同 api 镜像，启动 celery	—	异步任务：回测、日报、数据采集
celery_beat	同 api 镜像，启动 beat	—	定时任务调度器（每日触发）
freqtrade	freqtradeorg/freqtrade:stable	8080	回测/模拟盘/实盘执行引擎
flower	Celery 监控 UI	5555	查看任务队列状态
grafana	grafana/grafana	3000	策略性能可视化面板

1.3 核心原则：统一数据契约

最重要的工程决策： 所有数据，无论来自哪个框架或数据源，必须在进入 `services/` 层之前被转换成内部统一的 `Pydantic` 模型。`Freqtrade` 的回测结果和 `Jesse` 的回测结果，在平台内部看到的是同一个 `BacktestReport` 对象。

三个最关键的内部数据模型：

模型名	字段摘要	谁生产	谁消费
OHLCVBar	symbol/timeframe/open/high/low/close/volume/timestamp	CCXT Collector	Freqtrade/VectorBT/Jesse
BacktestReport	strategy_id/sharpe/profit_factor/max_dd/win_rate/expectancy/oos_sharpe	Freqtrade/Jesse Runner	GateKeeper/Strategy Library
RiskEvent	source/level(low/mid/high/critical)/description/affected_symbols/expires_at	News/Social Agent	Risk Engine/Execution Engine

二、开源框架集成方案

2.1 Freqtrade — 核心执行引擎（最高优先级）

定位： 回测引擎 + 模拟盘 + 实盘执行，覆盖验证层和执行层的 80% 工作，第一阶段主力框架

集成方式： 进程封装 + REST API 通信

Freqtrade 以独立 Docker 服务运行，平台通过其 REST API（port 8080）与之通信，不在 Python 进程内直接 import freqtrade。

freqtrade_runner.py 核心接口

```
class FreqtradeRunner:
    async def run_backtest(strategy_id: str, pairs: list, timerange: str) -> BacktestReport
    async def start_dryrun(strategy_id: str) -> str # 返回 run_id
    async def get_dryrun_status(run_id: str) -> DryrunStatus
    async def start_live(strategy_id: str) -> str
    async def get_open_trades() -> list[Trade]
    async def force_stop(strategy_id: str) -> bool
```

Freqtrade 配置管理

每个策略对应一份独立的 config.json，由平台动态生成，存放在 infra/freqtrade/config/{strategy_id}.json:

```
{
  "max_open_trades": 1,
  "stake_currency": "USDT",
  "stake_amount": 50,
  "stoploss": -0.02,           ← 平台风控层注入，策略不得覆盖
  "max_single_pair_stake_ratio": 0.1,
  "exchange": { "name": "binance", ... },
  "freqai": { ... }           ← 仅在使用 ML 策略时开启
}
```

策略代码存放规则

- 所有 Freqtrade 策略文件统一存放在 infra/freqtrade/strategies/
- 命名规则：{strategy_id}_{version}.py，例如 BTC_FakeBreakdown_v2.py
- 每个策略文件由 Coding Agent 生成，人工审核后合并
- 策略文件不得包含风控逻辑（止损大小由平台统一注入 config.json）

2.2 Jesse — 回测交叉验证引擎

定位： 零 look-ahead bias 回测引擎，用于验证 Freqtrade 回测结果的可信度，MIT 许可证（保护私有策

略)

集成方式: Python 子进程

Jesse 作为本地 Python 子进程运行, 不需要独立 Docker 服务。平台通过 subprocess 调用 jesse backtest 命令, 解析输出结果。

使用场景 (精准触发, 不滥用)

- Freqtrade 回测 Sharpe 在 1.0~1.5 之间 (边界策略), 用 Jesse 做第二次验证
- 策略从 Freqtrade 模拟盘晋升为实盘前, 必须通过 Jesse 样本外验证
- 怀疑 Freqtrade 回测存在 look-ahead bias 时触发 Jesse 复现

2.3 VectorBT — 大规模参数扫描

定位: 比 Freqtrade Hyperopt 快 100 倍的向量化回测引擎, 专用于参数空间扫描 (不用于最终回测报告)

典型使用场景

```
# 扫描 EMA 周期 5-200 × ATR 倍数 1.0-3.0 的全参数空间
# 在 BTC 3 年数据上运行, VectorBT 约需 30 秒
# Freqtrade Hyperopt 完成同样工作约需 6 小时
```

工作流程: VectorBT 扫描出最优参数区间 → 将参数区间传给 Freqtrade Hyperopt 做精细优化 → 最终参数固化进策略文件。两者串联, 不替代。

2.4 vnpy — A 股/期货执行层 (第三阶段)

定位: 第三阶段扩展 A 股市场时使用, 连接 CTP (期货) / 东方财富 (A 股) 接口。与你现有 A 股量化系统无缝衔接

集成方式: 进程封装。vnpy 作为独立服务运行, 平台通过 RPC/REST 与之通信, 接口设计与 freqtrade_bridge.py 保持一致 (同一套 Trade/Order 数据模型)。

现有 A 股量化系统的迁移策略:

- 直接移植: 已有的 vnpy 策略文件无需修改, 迁移到 infra/vnpy/strategies/
- 数据层打通: A 股历史数据写入同一个 TimescaleDB, 与加密货币数据共用数据基础设施
- 策略库统一: A 股策略进入同一个 Strategy Library, source 字段标注为 "A 股-自有"

2.5 QLib — WorldQuant Alpha 因子验证

定位： 微软开源 ML 量化平台，用于验证 WorldQuant Alpha 在 A 股/美股中的统计显著性，第二阶段引入

WorldQuant Adapter 核心工作

这是整个仓库最有差异化价值的模块。核心工作是将 WorldQuant Brain 的 Alpha 表达式转换成可回测代码：

Alpha 表达式（WQ 格式）	转换目标	转换方式
rank(close/delay(close,5))	QLib Factor → Freqtrade 信号	Coding Agent 辅助转换
correlation(close, volume, 20)	VectorBT 向量化因子	直接映射 pandas 操作
ts_mean(returns, 20) / ts_std(returns, 20)	Sharpe-like 动量因子	TA-Lib 等价实现

worldquant.py 核心逻辑

```
class WorldQuantAdapter:
    def parse_alpha_expression(expr: str) -> AlphaPlan
    def to_freqtrade_indicator(plan: AlphaPlan) -> str # 生成 Python 代码字符串
    def to_qlib_factor(plan: AlphaPlan) -> QlibFactor
    def batch_import(alpha_list: list[str]) -> list[Strategy] # 批量入库
```

2.6 QuantConnect/Lean — 美股/期货回测（第三阶段）

定位： 扩展至纳指、黄金、美股期权时使用，Lean 本地部署，数据接入 IQFeed 或 Polygon.io

集成方式： Docker 化 Lean 引擎，平台通过 lean-cli 命令行调用，解析 JSON 格式回测结果。接口与 FreqtradeRunner 保持统一（均输出 BacktestReport 对象）。

三、五层数据源完整接入方案

数据存储架构： TimescaleDB（PostgreSQL 超集）替代普通 PostgreSQL，专为时序数据优化，K 线查询性能提升 10-100 倍。Redis 只做热数据缓存（最新 1000 根 K 线），冷数据全在 TimescaleDB。

3.1 数据库 Schema（完整版）

TimescaleDB 超表（时序数据）

```
CREATE TABLE ohlcvBars (
    time          TIMESTAMPTZ NOT NULL,
    symbol        VARCHAR(20) NOT NULL,  -- BTC/USDT
    exchange      VARCHAR(20) NOT NULL,  -- binance
    timeframe     VARCHAR(10) NOT NULL,  -- 1h
    open NUMERIC, high NUMERIC, low NUMERIC, close NUMERIC, volume NUMERIC
);
SELECT create_hypertable('ohlcvBars', 'time');

CREATE TABLE marketExtras ( -- Funding Rate / OI / Long-Short
    time TIMESTAMPTZ NOT NULL, symbol VARCHAR(20),
    funding_rate NUMERIC, open_interest NUMERIC,
    long_ratio NUMERIC, short_ratio NUMERIC, liquidation_usd NUMERIC
);

CREATE TABLE riskEvents ( -- C/D 级数据生成的风险事件
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    source VARCHAR(50),    -- jinshi / twitter / macro_calendar
    level VARCHAR(20),     -- low / mid / high / critical
    description TEXT,
    affected_symbols TEXT[], -- NULL = 全市场
    expires_at TIMESTAMPTZ
);
```

3.2 A 级 — 市场数据接入（最高优先级，第一阶段完成）

OHLCV K 线：CCXT 统一接入

功能	实现方式	代码位置
历史数据下载	ccxt.binance().fetch_ohlcv() 分页拉取	services/data/collectors/market/ohlcv_downloader.py
实时 K 线流	ccxt.binance().watch_ohlcv() WebSocket	services/data/collectors/market/ohlcv_stream.py
数据完整性验证	检测缺口率>1%时报警并补数	services/data/storage/gap_checker.py

多交易所支持	exchange 参数化，同一接口切换 OKX/Bybit	services/data/collectors/market/ohlcv_downloader.py
--------	-------------------------------	---

加密特有数据：Funding Rate / OI / 清算

数据类型	API 来源	更新频率	存储表
Funding Rate	ccxt.fetch_funding_rate_history()	每 8 小时	market_extras
Open Interest	ccxt.fetch_open_interest_history()	每小时	market_extras
Long/Short Ratio	Binance /futures/data/globalLongShortAccountRatio	每 5 分钟	market_extras
清算数据 (Liquidation)	Binance /futures/data/forceOrders	实时	liquidations
Order Book (深度)	ccxt.watch_order_book() WebSocket	实时	仅 Redis，不落库

重要： Order Book 数据量极大（每秒更新），只在 Redis 里保留最新快照，不写 TimescaleDB。策略如需历史 Order Book，使用 Tardis.dev API（付费但最完整）。

.env.example 新增 A 级数据源密钥

```
# A 级市场数据
BINANCE_API_KEY=
BINANCE_API_SECRET=
OKX_API_KEY=           ← 新增，备用交易所
BYBIT_API_KEY=         ← 新增，备用交易所
```

3.3 B 级 — 宏观事件接入

经济日历：Trading Economics API

事件类型	数据源	处理逻辑	存储
FOMC 利率决议	Trading Economics / Forex Factory RSS	发布前 2h 标记 RISK=HIGH	macro_events 表
CPI/PPI/非农	同上	发布前 30min 暂停开仓	macro_events 表
美股重要财报	Alpha Vantage（免费层）	相关标的风险等级提升	macro_events 表
ETF 审批进展	CoinDesk RSS + 人工标记	正面/负面手动分类	macro_events 表

```
# B 级宏观数据
TRADING_ECONOMICS_API_KEY= ← 免费层够用
ALPHA_VANTAGE_API_KEY=     ← 免费层够用
```

风控触发逻辑（代码级别）

```
class MacroEventHandler:
    def on_event(event: MacroEvent):
        if event.impact == "HIGH" and event.minutes_until < 120:
            risk_engine.set_position_limit(multiplier=0.5)
        if event.impact == "HIGH" and event.minutes_until < 30:
            risk_engine.halt_new_entries(duration_minutes=90)
```

3.4 C 级 — 新闻数据接入

金十数据（优先级最高，国内最快）

金十数据没有官方 API，接入方案采用 RSS + WebSocket 双通道：

- 方案一（推荐）：使用 市场大事件 APP 的 RSS Feed（免费），轮询频率 30 秒
- 方案二（备用）：Selenium 无头浏览器监听页面更新（稳定性较差）
- 方案三（未来）：如果平台规模扩大，购买金十的付费数据接口

其他新闻源接入

数据源	接入方式	更新频率	费用
CoinDesk	官方 RSS Feed	近实时	免费
The Block	官方 RSS Feed	近实时	免费
Reuters （加密版块）	RSS Feed	近实时	免费
Bloomberg （加密版块）	RSS Feed（部分免费）	近实时	部分付费
SEC Filing	EDGAR RSS + sec-edgar-api	实时	免费
东方财富（A 股）	eastmoney-proxy 开源库	15 分钟延迟	免费

News Agent 处理流程

```
RSS/WebSocket → RawNews
↓
Claude API (News Agent)
Prompt: "分类以下新闻，返回 JSON: {category, risk_level, affected_symbols}"
↓
RiskEvent 写入 risk_events 表
↓
Risk Engine 实时订阅，自动调整仓位上限
```

```
# C 级新闻数据
JINSHI_RSS_URL=https://rss.jin10.com/flash_newest.xml ← 或第三方镜像
```

3.5 D 级 — 社交媒体接入

Twitter/X API（最重要但最贵）

Twitter API v2 Basic Tier（约 \$100/月）是最低可用档位，支持实时 Filtered Stream。监控账号白名单存储在数据库，可动态增减。

监控账号	原因	风险响应
@realDonaldTrump	贸易/监管政策	检测到关税/crypto 关键词→RISK=HIGH，仓位减半
@elonmusk	DOGE/BTC 情绪	检测到 crypto 关键词→记录，不自动触发（噪声高）
@cz_binance	交易所安全	检测到"paused/suspended"→RISK=CRITICAL，暂停交易
@VitalikButerin	ETH 生态	检测到重大协议更新→记录，人工确认
@federalreserve	货币政策	非正式表态→RISK=HIGH，仓位降 40%

```
# D 级社交媒体
TWITTER_BEARER_TOKEN=      ← Basic Tier $100/月
TWITTER_WATCH_USER_IDS=    ← 逗号分隔的用户 ID 列表
```

Telegram 信号频道

```
# Telegram
TELEGRAM_BOT_TOKEN=        ← 自建 Bot，监听目标频道
TELEGRAM_CHANNEL_IDS=      ← 逗号分隔的频道 ID
```

Telegram Agent 解析逻辑（不依赖固定格式，AI 解析）：

```
输入: "BTC Long entry 65000 SL 63000 TP1 68000 TP2 72000 leverage 5x"
输出: TradeSignal(symbol="BTC", side="long", entry=65000, sl=63000,
                  tp=[68000, 72000], leverage=5, source="channel_xyz")
```

3.6 E 级 — 策略研究数据接入

GitHub 策略爬取

```
class GitHubStrategyResearcher:
    # 搜索高星 Freqtrade/Jesse 策略仓库
    search_queries = [
        "freqtrade strategy stars:>50",
        "jesse trading strategy crypto",
        "backtrader strategy crypto",
    ]
    def crawl_and_summarize(repo_url) -> StrategySummary
    # Research Agent 读取代码 → 提炼核心逻辑 → 写入策略库
```

```
# E 级研究数据
GITHUB_TOKEN=          ← 免费，提高 API 限额
ARXIV_CATEGORIES=q-fin.TR,q-fin.PM ← 量化金融论文
```

四、分阶段开发任务清单

Phase 0（第 1-3 周）：基础设施真正跑起来

目标： docker-compose up 一键启动全部服务，BTC/USDT 三年历史数据写入 TimescaleDB，第一个策略代码能跑通回测。

任务 ID	任务描述	负责层	完成标准	优先级
P0-01	将 PostgreSQL 换成 TimescaleDB，更新 docker-compose + init.sql	infra	TimescaleDB 服务正常启动，ohlcvBars 超表创建成功	P0（阻塞后续）
P0-02	添加 celery_worker + celery_beat + flower 服务到 docker-compose	infra	flower UI 可访问，测试任务可正常入队和执行	P0（阻塞后续）
P0-03	实现 ohlcv_downloader.py：CCXT 拉取 BTC/ETH/SOL 历史数据	data/collectors	3 年分钟级数据完整下载，缺口率<1%	P0（最先做）
P0-04	实现 gap_checker.py：数据完整性验证	data/storage	能检测并报告数据缺口	P0
P0-05	实现 Strategy SQLAlchemy 模型（18 字段）+ Alembic migration	strategy_library	migrate 成功，可从 psql 查询空策略表	P0
P0-06	补全 .env.example（所有数据源密钥占位符）	infra	文件包含本文档列出的全部 KEY	P0
P0-07	添加 freqtrade 服务到 docker-compose，配置 BTC/USDT 基础 config.json	infra	freqtrade webUI 8080 可访问，API 正常响应	P0
P0-08	实现第一个 Freqtrade 策略：EMA 趋势跟随 + ATR 止损	validation/engines	freqtrade backtesting 返回回测结果 JSON	P0
P0-09	实现 report_parser.py：解析 Freqtrade JSON → BacktestReport 对象	validation	能正确解析 Sharpe/PF/MaxDD/WinRate	P0
P0-10	实现 gate_keeper.py：门槛检查	validation	Sharpe<1.0 的策略被正确拒绝	P0
P0-11	补全 project-memory.md / decisions-log.md / task-history.md	memory	三文件有实质内容，AI 开工前可读取定向	P0
P0-12	实现最基础的 FastAPI 路由：GET /strategies, POST /strategies	apps/api	Swagger UI 可访问，CRUD 操作正常	P0

Phase 1（第 4-8 周）：策略库 + 回测流水线打通

目标： 任何一个新策略想法，从自然语言到回测报告，能在一天内走完全程。WorldQuant Alpha 前 100 条入库。

任务 ID	任务描述	负责层	完成标准
P1-01	实现 Strategy Agent: 自然语言 → 策略规则 JSON	agents	给定"假突破做空"描述，输出包含精确入场条件的 JSON
P1-02	实现 Coding Agent: 策略规则 JSON → Freqtrade Python 代码	agents	生成的代码能直接被 Freqtrade 加载运行
P1-03	实现 worldquant.py Adapter: Alpha 表达式批量导入	research_source	前 100 条 Alpha 成功写入 strategy_library 表
P1-04	实现 VectorBT Scanner: 参数扫描	validation/engines	给定 EMA 周期范围，能生成热力图显示最优参数区间
P1-05	实现 ohlcv_stream.py: WebSocket 实时 K 线入库	data/collectors	BTC 1m K 线实时写入，延迟<2 秒
P1-06	实现 market_extras 采集: Funding Rate + OI + L/S Ratio	data/collectors	每 8h/每 5min 自动更新
P1-07	实现 freqtrade_runner.py 完整版 (REST API 通信)	validation/engines	能通过 API 触发回测、启停模拟盘
P1-08	实现 jesse_runner.py (子进程封装)	validation/engines	给定策略，能运行 Jesse 回测并返回 BacktestReport
P1-09	实现 Celery 任务: daily_data_sync (每日凌晨数据同步)	tasks	每天 2AM 自动运行，同步前日完整 K 线数据
P1-10	实现 github_crawler.py: 爬取高星策略并交 Research Agent 总结	research_source	每周自动扫描，新策略草稿写入策略库
P1-11	前端: 策略库列表页面 (React)	frontend	能查看所有策略、回测状态、关键指标
P1-12	将现有 A 股量化系统策略整理入库	strategy_library	A 股策略以统一格式完整入库，source="A 股-自有"

Phase 2（第 9-14 周）：模拟盘 + 风控 + 数据层 C/D 级

目标： 通过回测门槛的策略自动上线模拟盘，风控系统上线，新闻和社媒数据实时影响仓位上限。

任务 ID	任务描述	负责层	完成标准
P2-01	实现 Risk Engine 核心: 仓位上限/日亏损停止/连亏保护	execution	三类风控规则触发测试通过

P2-02	实现 Execution Engine：下单/止损/日志	execution	模拟盘能正确执行开平仓，日志完整
P2-03	实现 News Agent + 新闻 RSS 采集（金十/CoinDesk 等）	agents+data	新闻收到 30 秒内生成 RiskEvent，写入 DB
P2-04	实现 MacroEventHandler：经济日历→风控参数调整	data/collectors	FOMC 当天仓位上限自动降 50%
P2-05	实现 Twitter Agent + API 接入（重点账号监控）	agents+data	特朗普发关税词汇→5 分钟内风险等级提升
P2-06	实现 Telegram Agent + Bot 接入	agents+data	喊单信号自动解析写入 DB，开始统计胜率
P2-07	实现 review_agent.py：每日日报生成	review	每天 22:00 自动生成日报，包含所有策略表现
P2-08	实现 failure_analyzer.py + strategy_updater.py	review	失效策略自动标注，failure_reasons 字段更新
P2-09	Freqtrade 模拟盘自动化：通过门槛→自动部署模拟盘	validation→execution	手动触发或 Review Agent 推荐后一键启动模拟盘
P2-10	前端：日报展示页 + 风控状态面板 + 模拟盘监控	frontend	能实时查看当日风控状态和模拟盘持仓
P2-11	Grafana 面板：策略表现曲线/回撤图/风险事件时间轴	infra/grafana	三个核心面板可用

Phase 3（第 15 周+）：小资金实盘 + 多市场扩展

目标： 10-50 USDT 小资金实盘验证执行环境，A 股/美股数据层扩展，Lean 引擎接入。

任务 ID	任务描述	负责层
P3-01	Freqtrade 切换为真实交易模式（live mode），金额上限 10U	execution
P3-02	freqtrade_bridge.py 扩展：实盘仓位同步到平台 DB	execution
P3-03	接入 vnpy，A 股数据写入 TimescaleDB，现有 A 股策略上线模拟盘	data+execution
P3-04	Lean 引擎 Docker 化，接入纳指/黄金回测	validation/engines
P3-05	QLib Factor 验证流程：WQ Alpha → QLib 有效性检验	research_source
P3-06	Optimization Agent：自动触发 Hyperopt（基于 Review 建议）	agents
P3-07	Research Agent：定时扫描 arXiv q-fin.TR 新论文	agents

五、关键文件模板（直接可用）

5.1 完整 .env.example

所有数据源密钥占位符，按分级组织：

```
# — 平台基础配置 —————
APP_ENV=development
APP_NAME=ai-quant-research-platform
API_HOST=0.0.0.0
API_PORT=8000

# — 基础设施 —————
POSTGRES_URL=postgresql://postgres:postgres@timescaledb:5432/ai_quant
REDIS_URL=redis://redis:6379/0
CELERY_BROKER_URL=redis://redis:6379/1
CELERY_RESULT_BACKEND=redis://redis:6379/2

# — AI —————
CLAUDE_API_KEY=
CLAUDE_MODEL=claude-sonnet-4-6

# — A 级：加密交易所 —————
BINANCE_API_KEY=
BINANCE_API_SECRET=
OKX_API_KEY=
OKX_API_SECRET=
OKX_PASSPHRASE=
BYBIT_API_KEY=
BYBIT_API_SECRET=
DEFAULT_EXCHANGE=binance

# — B 级：宏观数据 —————
TRADING_ECONOMICS_API_KEY=
ALPHA_VANTAGE_API_KEY=
FOREXFACTORY_RSS_URL=https://forexfactory.com/calendar.rss

# — C 级：新闻数据 —————
JINSHI_RSS_URL=https://rss.jin10.com/flash_newest.xml
COINDESK_RSS_URL=https://www.coindesk.com/arc/outboundfeeds/rss/
THEBLOCK_RSS_URL=https://www.theblock.co/rss.xml
REUTERS_CRYPTOCURRENCY_RSS_URL=
SEC_EDGAR_RSS_URL=https://efits.sec.gov/LATEST/search-index?q=%22crypto%22&dateRange=custom

# — D 级：社交媒体 —————
TWITTER_BEARER_TOKEN=
TWITTER_WATCH_USER_IDS=25073877,44196397,902926941413453824
TELEGRAM_BOT_TOKEN=
```

```
TELEGRAM_CHANNEL_IDS=

# — E 级：研究数据 —————
GITHUB_TOKEN=
ARXIV_CATEGORIES=q-fin.TR,q-fin.PM
WORLDQUANT_BRAIN_SESSION= ← 本地 Alpha 文件路径（不上传密钥）

# — Freqtrade —————
FREQTRADE_API_URL=http://freqtrade:8080
FREQTRADE_USERNAME=freqtradeuser
FREQTRADE_PASSWORD=
```

5.2 Makefile（统一命令入口）

```
# 环境管理
make up          # docker-compose up -d（启动全部服务）
make down        # docker-compose down
make logs        # 查看所有服务日志

# 数据管理
make data-sync   # 触发历史数据同步（一次性）
make data-check  # 检查数据完整性

# 策略开发
make backtest STRATEGY=EMA_Trend_v1 # 单策略回测
make backtest-all                # 全策略批量回测
make scan STRATEGY=EMA_Trend_v1   # VectorBT 参数扫描

# 测试
make test          # pytest 全量
make test-unit     # 仅单元测试（快）

# 记忆维护
make memory-update # 提示更新三个记忆文件
```

5.3 project-memory.md 模板（AI 协作记忆文件）

```
# AI Quant Research Platform — 项目记忆

## 当前阶段
Phase 0 — 基础设施

## 已完成
- [ ] TimescaleDB 替换 PostgreSQL
- [ ] Celery Worker 服务添加
- [ ] BTC/ETH/SOL 历史数据下载
```

```

## 核心设计决策
- Freqtrade 以独立 Docker 服务运行，通过 REST API 通信（不直接 import）
- 统一数据契约：所有框架输出必须转换为内部 Pydantic 模型
- TimescaleDB 替代 PostgreSQL（时序数据性能）
- WorldQuant Adapter 是最高差异化价值模块，第二阶段优先完成

## 已知风险
- Twitter API $100/月成本，Phase 2 再接入
- Jesse runner 目前是子进程调用，与 Docker 环境的路径问题待解决

## 下一个任务
P0-01: 更换 TimescaleDB

```

5.4 Agent Base 类模板

```

class BaseAgent:
    """所有 Agent 的基类，封装 Claude API 调用"""
    model = "claude-sonnet-4-6"
    max_tokens = 4096

    def __init__(self, system_prompt: str):
        self.system_prompt = system_prompt
        self.client = anthropic.AsyncAnthropic()

    async def run(self, user_message: str, context: dict = None) -> str:
        messages = [{"role": "user", "content": user_message}]
        if context:
            messages[0]["content"]
            f"Context:\n{json.dumps(context)}\n\n{user_message}"
        response = await self.client.messages.create(
            model=self.model,
            max_tokens=self.max_tokens,
            system=self.system_prompt,
            messages=messages,
        )
        return response.content[0].text

```

六、优先级矩阵与风险管控

6.1 任务优先级决策框架

当任务并行时，按以下优先级顺序决策：

优先级	判断条件	示例任务
P0 阻塞型	后续所有任务依赖此任务完成	TimescaleDB 替换、Celery 添加、数据下载
P1 核心路径	在研究闭环关键路径上	Strategy Library 模型、Freqtrade Runner、回测报告解析
P2 增强型	提升效率但不阻塞主流程	VectorBT 扫描、Jesse 交叉验证、GitHub 爬取
P3 扩展型	多市场/多框架扩展	Lean 接入、vnpy A 股、QLib 因子验证

6.2 框架集成风险与规避方案

风险	触发条件	规避方案
Freqtrade API 版本不兼容	升级 Freqtrade 后 REST API 字段变更	固定 Freqtrade Docker 镜像版本号，升级前在 dev 环境测试
Jesse 子进程路径问题	Docker 内路径和本地路径不一致	Jesse 单独运行在开发机器，不 Docker 化；Phase 2 再解决
TimescaleDB 数据量爆增	高频 Order Book 数据误写入 DB	严格区分 Redis only 数据和 DB 数据，Order Book 禁止落库
Twitter API 成本	\$100/月 是固定支出	Phase 2 才接入；Phase 0-1 改用免费 RSS 新闻代替
CCXT 限流	高频调用被交易所封禁	Redis 缓存实时数据，WebSocket 替代轮询，每个交易所限速配置
Celery 任务积压	回测任务过多导致队列阻塞	回测任务单独 queue（backtest_queue），与日常数据同步隔离
WorldQuant Adapter 转换错误	Alpha 表达式转换后逻辑错误	每条 Alpha 必须有单元测试，生成代码后 dry-run 验证输出

6.3 三个最重要的工程原则（必须坚守）

原则一：框架内聚 Freqtrade 相关代码全部在 `services/validation/engines/freqtrade_runner.py` 和 `infra/freqtrade/` 内。`apps/api/` 只知道 `BacktestReport`，不知道 `Freqtrade` 的存在。

原则二：数据契约优先 新增任何数据源，先定义 Pydantic 模型，再写采集代码。模型定义在 `shared/models/` 目录，所有 `services/` 都从这里 `import`，不要各自定义。

原则三：记忆文件随代码更新 每次完成一个 Phase 任务，必须同步更新 `project-memory.md` 和 `decisions-log.md`。AI 协作者的上下文完全依赖这两个文件，不更新等于让 AI 失忆。

6.4 第一个工作日的精确行动序列

按这个顺序执行，保证 Phase 0 无卡点：

1. `git clone` 仓库，`docker-compose up -d` 验证当前三服务可启动
2. 修改 `docker-compose.yml`：postgres 镜像改为 `timescaledb/timescaledb:latest-pg16`，添加 `celery_worker` + `celery_beat` + `flower` + `freqtrade` 四个服务
3. 创建 `infra/timescale/init.sql`，定义 `ohlcvBars` 超表和 `market_extras` 超表
4. `docker-compose down && docker-compose up -d`，验证全部 8 个服务正常启动
5. 写 `services/data/collectors/market/ohlcv_downloader.py`，用 `ccxt` 下载 BTC/USDT 过去 3 年 1h 数据写入 TimescaleDB
6. 运行下载脚本，验证数据条数（ $3 \text{ 年} \times 24 \times 365 \approx 26,000$ 条）
7. 更新 `project-memory.md`：P0-01、P0-02、P0-03 标记为完成
8. 开始 P0-05：创建 Strategy SQLAlchemy 模型文件，运行 Alembic migration

第一周结束的可见标准： `docker-compose up` 启动 8 个服务无报错 + TimescaleDB 中有 BTC/ETH/SOL 历史数据 + `strategy_library` 表结构创建成功 + 至少一个 Freqtrade 策略能跑通回测。这四条全部满足，Phase 0 完成。